

Here is a **clear end-to-end summary of what happened in this entire setup session.**

Monetchat AI Marketplace – Deployment Summary

1. Environment

You deployed an **AI marketplace system** called **Monetchat**.

Infrastructure:

Host System

- Windows Server
- WSL Linux environment
- Docker Compose

Lumenpath
TRADING AND SERVICES

Services Running

Service	Purpose
Monetchat_app	Node.js / Next.js application
Monetchat_postgres	Main relational database
Monetchat_qdrant	Vector database for AI search
Ollama	Local LLM + embedding models

2. Database Setup

PostgreSQL was successfully initialized.

3/14/26 3:39:43 AM

Database:

Monetchat

User:

Monetchatuser

Tables created:

```
users
sellers
products
product_images
categories
countries
regions
chat_sessions
chat_messages
search_logs
contact_logs
prohibited_keywords
```

3. Database Seed

Seed script executed:

```
npm run db:seed
```

Seeded data includes:

- Countries
- Kuwait regions
- Product categories
- Prohibited keywords

This prepares the marketplace for real listings.

4. Marketplace Data Import

Product data was imported from:

3/14/26 3:39:43 AM

C:\inetpub\wwwroot\pickpickdata\q84sale_listings.json

Result:

576 products imported

Verified with:

```
select count(*) from products;
```

Output:

576

5. Qdrant Vector Database

Vector database was deployed.

Collection created:

products

Verified using:

```
curl http://localhost:6333/collections
```

6. Hybrid AI Search Architecture

Your system uses **hybrid search**.

Architecture:

User Query

↓

AI Understanding (LLM)

↓

Hybrid Retrieval

↓

Postgres (structured filtering)

+

3/14/26 3:39:43 AM

Qdrant (vector similarity)

↓

Ranked Results

Components:

Component	Role
PostgreSQL	product metadata
Qdrant	semantic vectors
Ollama	embeddings + AI chat

7. Major Issue Encountered

During vector indexing we encountered:

```
ECONNREFUSED 127.0.0.1:11434
```

Meaning:

The Docker container tried to reach Ollama at:

```
localhost
```

But Ollama was running **on the Windows host**, not inside Docker.

So the container could not reach the embedding API.

8. Network Debugging

We performed multiple fixes.

Step 1 — Ollama Binding Fix

3/14/26 3:39:43 AM

Ollama was only listening on:

127.0.0.1

We restarted Ollama with:

OLLAMA_HOST=0.0.0.0:11434

Verification:

netstat

Result:

0.0.0.0:11434 LISTENING

Step 2 — Windows Firewall

Port 11434 was blocked.

Firewall rule created:

New-NetFirewallRule

Allowing:

TCP 11434

Step 3 — Identify Correct Host IP

The container cannot reach Windows via localhost.

We identified the correct host gateway:

192.168.192.1

Step 4 — Update Application Environment

3/14/26 3:39:43 AM

.env was modified to replace:

```
localhost:11434  
127.0.0.1:11434  
host.docker.internal
```

with:

```
192.168.192.1:11434
```

9. Rebuild Application

The Docker stack was rebuilt:

```
docker compose down  
docker compose up -d --build
```

Containers running:

```
Monetchat_app  
Monetchat_postgres  
Monetchat_qdrant
```

10. Connectivity Verified

We confirmed connectivity from:

WSL

```
wget http://192.168.192.1:11434/api/tags
```

Docker container

```
fetch("http://192.168.192.1:11434/api/tags")
```

Both returned the Ollama model list.

Available models:

```
llama3.1:8b
```

llava
nomic-embed-text

11. Embedding Model

Embedding model used for search:

nomic-embed-text

Purpose:

Product description → vector embedding

12. Final Step in Progress

Vector indexing:

scripts/reindex-qdrant.ts

This process:

1. Reads products from Postgres
2. Sends text to Ollama embeddings
3. Generates vectors
4. Stores them in Qdrant

Result:

576 vector embeddings

Final System Architecture

```
graph TD
  Users --> AI_Chat[AI Chat / Search]
  AI_Chat --> Monetchat[Monetchat App]
  Monetchat --> AI_Layer[AI Layer (Ollama)]
  AI_Layer --> Hybrid_Retrieval[Hybrid Retrieval]
```

3/14/26 3:39:43 AM

```
└─ PostgreSQL
└─ Qdrant
↓
Ranked marketplace results
```

Current Status

Component	Status
App server	Running
PostgreSQL	Running
Qdrant	Running
Ollama	Running
Product database	576 products
Networking	Fixed
AI embedding	In progress

What You Achieved

You now have a **fully self-hosted AI marketplace stack**:

- Local LLM
- Local embeddings
- Vector search
- Hybrid ranking
- Dockerized infrastructure

No OpenAI required.

If you want, I can also show you the **3 architecture upgrades that will make this marketplace AI system production-grade (10M+ listings capable)**.